CSS Selectors - Complete Guide

What Are CSS Selectors?

CSS Selectors are patterns used to select and style HTML elements. They tell CSS *which elements* to apply styles to. Mastering selectors is key to writing efficient CSS.

1. Basic Selectors

Universal Selector (*)

Selects **all** elements on the page:

```
* {
  margin: 0;
  padding: 0;
}

/* Removes default margins and padding from all elements */
```

Element Selector

Selects all elements of a specific type:

```
p {
  color: blue;
}

h1 {
  font-size: 32px;
}

/* All <p> tags will be blue */
/* All <h1> tags will be 32px */
```

Class Selector (.)

Selects elements with a specific class. Use a period (.):

```
.highlight {
  background-color: yellow;
}

/* HTML */
<p class="highlight">This is highlighted</p>
<span class="highlight">This too</span>

/* Both will have yellow background */
```

Multiple classes on one element:

```
.primary {
  color: blue;
}

.large {
  font-size: 20px;
}

/* HTML */
<p class="primary large">Large blue text</p>
```

ID Selector (#)

Selects a single element with a specific ID. Use a hash (#):

```
#header {
  background-color: navy;
  color: white;
}

/* HTML */
<div id="header">Welcome to my site</div>

/* Only this specific element is styled */
```

Important: IDs should be **unique** on a page. Use for single, specific elements.

2. Attribute Selectors

Select elements based on their attributes:

Exact Match

```
input[type="text"] {
  border: 2px solid blue;
}

/* Selects all <input> tags with type="text" */
```

Attribute Contains

```
/* Attribute value contains "submit" */
input[value*="submit"] {
  background-color: green;
}

/* Selects input with value containing "submit" */
```

Attribute Starts With

```
/* Attribute value starts with "https" */
a[href^="https"] {
  color: green;
}

/* All secure links will be green */
```

Attribute Ends With

```
/* Attribute value ends with ".pdf" */
a[href$=".pdf"] {
  color: red;
}

/* All PDF links will be red */
```

Attribute Contains Word

```
/* Attribute contains specific word */
a[data-status~="active"] {
  font-weight: bold;
}

/* Matches exact word in space-separated list */
```

3. Pseudo-class Selectors (:)

Select elements in a specific **state** or **position**:

Link States

```
/* Unvisited link */
a:link {
  color: blue;
}

/* Visited link */
a:visited {
  color: purple;
}

/* Mouse hovering over link */
a:hover {
  color: red;
  text-decoration: underline;
}

/* Link being clicked */
a:active {
  color: orange;
}
```

User Interaction

```
/* Element has focus (keyboard or click) */
input:focus {
  outline: 3px solid blue;
  border-color: blue;
}

/* Mouse hovering over any element */
button:hover {
  background-color: darkblue;
}

/* Element is checked (checkbox, radio) */
input:checked {
  background-color: green;
}
```

Position-based

```
/* First child of parent */
li:first-child {
  font-weight: bold;
}

/* Last child of parent */
li:last-child {
  border-bottom: none;
}

/* nth child (count starts at 1) */
li:nth-child(2) {
  background-color: lightgray;
}

/* Every other item */
li:nth-child(odd) {
  background-color: #f0f0f0;
}

li:nth-child(even) {
  background-color: white;
}

/* First of its type */
p:first-of-type {
  font-size: 18px;
}

/* Only child */
p:only-child {
  color: blue;
}
```

Form States

```
/* Disabled form element */
input:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

/* Enabled form element */
input:enabled {
```

```
  cursor: text;
}

/* Input is required */
input:required {
  border-color: red;
}

/* Input has invalid value */
input:invalid {
  border-color: red;
}

/* Input has valid value */
input:valid {
  border-color: green;
}
```

Content States

```
/* Element has no children */
div:empty {
  display: none;
}

/* Text is selected */
p::selection {
  background-color: yellow;
  color: black;
}

/* Element matches selector */
li:is(.active, .highlight) {
  font-weight: bold;
}
```

4. Pseudo-element Selectors (::)

Select **parts** of an element:

```
/* First letter of element */
p::first-letter {
  font-size: 32px;
  font-weight: bold;
  color: blue;
```

```
}

/* First line of element */
p::first-line {
  text-transform: uppercase;
  color: gray;
}

/* Before element content */
p::before {
  content: →" ";
  color: red;
}

/* After element content */
p::after {
  content: " ←";
  color: red;
}

/* Selected/highlighted text */
p::selection {
  background-color: yellow;
}

/* Placeholder text in input */
input::placeholder {
  color: gray;
  font-style: italic;
}
```

5. Combinators

Combine multiple selectors to target elements based on their **relationships**:

Descendant Combinator (space)

Selects elements inside (at any depth):

```
div p {
  color: blue;
}

/* All <p> tags inside <div> will be blue */
/* Works at any depth */
```

```
<div>
  <p>Blue</p>
  <section>
    <p>Also blue</p>
  </section>
</div>
```

Child Combinator (>)

Selects direct children only:

```
div > p {
  color: green;
}

/* Only direct <p> children of <div> */

<div>
  <p>Green</p>
  <section>
    <p>NOT green (too deep)</p>
  </section>
</div>
```

Adjacent Sibling Combinator (+)

Selects the next sibling element:

```
h2 + p {
  font-weight: bold;
}

/* The first <p> right after <h2> */

<h2>Title</h2>
<p>Bold (directly after h2)</p>
<p>Not bold (not immediately after)</p>
```

General Sibling Combinator ( )

Selects all following siblings:

```
h2 ~ p {
  color: gray;
}

/* All <p> tags that come after <h2> */
```

```
<h2>Title</h2>
<p>Gray</p>
<span>Other element</span>
<p>Also gray</p>
```

6. Group Selector (,)

Style multiple selectors with one rule:

```
h1, h2, h3, h4, h5, h6 {
  font-family: Arial;
  color: darkblue;
}

/* All heading tags get same style */

p, span, div {
  margin: 10px;
}

/* Multiple elements, same rule */
```

7. Advanced Selectors

Not Selector (:not)

Select elements that do **NOT** match:

```
/* All paragraphs except those with class "ignore" */
p:not(.ignore) {
  color: blue;
}

/* All inputs except disabled ones */
input:not(:disabled) {
  cursor: text;
}
```

Has Selector (:has)

Select elements that **contain** matching elements:

```
/* Div that contains a paragraph */
div:has(p) {
```

```
  border: 2px solid blue;
}

/* List that has checked items */
ul:has(input:checked) {
  background-color: lightgreen;
}
```

Is Selector (:is)

Match any selector in the list:

```
/* Same as h1, h2, h3 { ... } */
:is(h1, h2, h3) {
  font-family: Arial;
}
```

Where Selector (:where)

Like `:is` but with zero specificity:

```
:where(h1, h2, h3) {
  font-family: Arial;
}
```

Selector Specificity

Understanding **specificity** helps when multiple rules apply:

Specificity Score (lower to higher):

```
Element selector:           1 point
Class selector:             10 points
ID selector:                100 points
Inline style:               1000 points
!important:                 highest priority

Examples:
p                           /* 1 point */
.highlight                  /* 10 points */
#header                     /* 100 points */
div > p.highlight           /* 1 + 10 = 11 points */
#main .container p          /* 100 + 10 + 1 = 111 points */
<p style="color: red;">     /* 1000 points */
```

Best Practices

1. Use **classes** for styling (reusable)
2. Use **IDs** for unique elements (JavaScript hooks)
3. Keep selectors **simple** and **specific**
4. Avoid overly complex combinators
5. Use `:is()` to reduce specificity
6. Don't rely on `!important`
7. Consider semantic HTML for easier selection

Common Patterns

```
/* Active navigation item */
nav a.active {
  color: orange;
  border-bottom: 3px solid orange;
}

/* Button on hover */
button:hover {
  background-color: darkblue;
}

/* Form validation */
input:invalid {
  border-color: red;
}

input:valid {
  border-color: green;
}

/* Alternate row colors */
tr:nth-child(even) {
  background-color: #f0f0f0;
}

/* Remove default styles */
ul li:first-child {
  list-style: none;
}
```

Quick Reference

Selector	Example
Universal	*
Element	p, div, h1
Class	.classname
ID	#idname
Attribute	[type=”text”]
Pseudo-class	:hover, :nth-child(2)
Pseudo-element	::before, ::after
Descendant	div p
Child	div > p
Adjacent sibling	h2 + p
General sibling	h2 ~ p
Group	h1, h2, h3